

Тема: «Объекты, свойства и методы»

Объект – это интегрированный набор данных, необходимых для описания его природы, и функций, при помощи которых на объект можно воздействовать.

Данные объекта называются свойствами объекта, а его функции – методами. Для обращения к свойству необходимо указать имена объекта и свойства, разделив их точкой. При использовании метода объекта нужно перед именем метода указать имя объекта, к которому он принадлежит. Например, правильным обращением к методу `document` является выражение `document.write()`, а просто выражение `write()` приведет к ошибке.

Все объекты JavaScript можно разделить на три группы:

- объекты браузера;
- внутренние или встроенные объекты языка JavaScript;
- объекты, связанные с тегами языка HTML;

Объекты браузера

Объекты верхнего уровня, или объекты браузера – это объекты, поддерживаемые в среде браузера: `window`, `location`, `history`, `document`, `navigator`. Объекты, перечисленные в таблице 1, создаются автоматически при загрузке документа в браузер.

Таблица 1 – Объекты браузера

Имя объекта	Описание
<code>window</code>	Объект верхнего уровня в иерархии объектов языка JavaScript. Фреймосодержащий документ также имеет объект <code>window</code> .
<code>document</code>	Содержит свойства, которые относятся к текущему HTML-документу, например, имя каждой формы, цвета, используемые для отображения документа и другое. В языке JavaScript большинству HTML-тегов соответствуют свойства объекта <code>document</code> .
<code>location</code>	Содержит свойства, описывающие местонахождение текущего документа, например, адрес URL.
<code>navigator</code>	Содержит информацию о версии браузера. Свойства данного объекта обычно только для чтения. Например, свойство <code>navigator.appname</code> содержит строковое значение имени браузера.
<code>history</code>	Содержит информацию обо всех ресурсах, к которым пользователь обращался во время текущего сеанса работы с браузером.

Далее более подробно будет рассмотрен каждый объект.

1 Объект Window

Объект `Window` обычно соответствует главному окну браузера и является объектом верхнего уровня в языке JavaScript, поскольку документы собственно и открываются в окне. В фреймосодержащих документах, объект `window` может не всегда соответствовать главному окну программы. Поэтому для обращения к конкретному окну следует использовать свойство `frames` объекта `parent`. Фреймы – это те же окна. Чтобы обратиться к ним в языке JavaScript, можно использовать массив `frames`. Например, выражение `parent.frames[0]` обращается к первому фрейму окна браузера. Предполагается, что такое окно существует, но при помощи метода `window.open()` можно открывать и другие окна и обращаться к ним посредством свойств объекта `window`.

Для обращения к методам и свойствам объекта `window` используют следующие варианты записи:

- `window.propertyName`

- `window.methodName(parameters)`
- `self.propertyName`
- `self.methodName(parameters)`
- `top.propertyName`
- `top.methodName(parameters)`
- `parent.propertyName`
- `parent.methodName(parameters)`
- `windowVar.propertyName`
- `windowVar.methodName(parameters)`
- `propertyName`
- `methodName(parameters)`

Здесь `windowVar` – экземпляр объекта `window`. Имя `self` – синоним для обращения к текущему окну во фреймосодержащем документе, тогда как имя `top` применяется для обращения к главному окну браузера. Для этой цели можно применить и объект `parent`. Однако следует иметь в виду, что значением `parent` является ссылка на родительское окно, когда `top` – ссылка на окно верхнего уровня, которое содержит либо данный фрейм, либо фрейм, содержащий вложенный фрейм. Обращение напрямую к методам и свойствам возможно при использовании оператора `with`.

1.1 Основные свойства объекта `window`

Объект `window` имеет следующие свойства:

`defaultStatus` – текстовое сообщение, которое по умолчанию выводится в строку состояния окна браузера;

`frames` – массив фреймов во фреймосодержащем документе;

`length` – количество фреймов во фреймосодержащем документе;

`name` – заголовок окна, который задается с помощью аргумента `windowName` метода `open()`;

`parent` – синоним для обращения к родительскому окну;

`self` – синоним, используемый для обращения к текущему окну браузера;

`status` – текст временного сообщения в строке состояния окна браузера;

`top` – синоним, используемый для обращения к главному окну браузера;

`window` – синоним, используемый для обращения к текущему окну браузера.

1.2 Основные методы объекта `window`

Метод `alert()` применяется для того чтобы вывести на экран текстовое сообщение. Для открытия окна используется метод `open()`, а для закрытия – метод `close()`. С помощью метода `confirm()` происходит вывод на экран окна сообщения с кнопками `Yes` и `No`, и возвращает булево значение `true` или `false` в зависимости от нажатой кнопки. С помощью метода `prompt()` на экран выводится диалоговое окно с полем вывода. Метод `setTimeout()` устанавливает в текущем окне обработку событий, связанных с таймером, а метод `clearTimeout()` отменяет обработку таких событий. Подробное описание свойств и методов объекта `window` представлено в таблице 2.

Таблица 2 – Свойства и методы объекта `window`

Тип	Элемент	Описание
Метод	<code>atob()</code>	Декодирование строки, закодированной с помощью с помощью кода <code>base 64</code>

JS. Объекты браузера

	alert()	Выводит диалоговое окно предупреждения, отображающее переданную в метод текстовую строку.
	back()	Загружает предыдущую страницу вместо экземпляра window.
	blur()	Убирает фокус из окна.
	btoa()	Кодирует строку с использованием кода base 64
	captureEvents()	Заставляет окно перехватывать все события указанного типа.
	clearInterval()	Обнуляет интервал, установленный методом setInterval
	clearTimeout()	Удаляет тайм-аут, установленный методом setTimeout()
	close()	Закрывает экземпляр окна.
	confirm()	Отображает диалоговое окно подтверждения.
	crypto.random()	Генерирует случайную строку данных заданной длины
	crypto.signText()	Возвращает строку закодированных данных, которая представляет указанный объект.
	disableExternalCapture()	Запрещает внешний перехват событий.
	enableExternalCapture()	Разрешает внешний перехват событий для страниц, загруженных с других серверов
	find()	Отображает диалоговое окно поиска, в которое можно вводить текст для поиска на текущей странице.
	focus()	Устанавливает фокус на указанное окно.
	forward()	Загружает следующую страницу вместо экземпляра window.
	handleEvent()	Вызывает обработчик для передаваемого события.
	home()	Загружает определенную пользователем страницу в экземпляр window.
	moveBy()	Перемещает окно в соответствии с заданным смещением
	moveTo()	Перемещает окно в заданную позицию.
	open()	Открывает новый экземпляр окна.
	print()	Вызывает диалоговое окно печати, позволяющее вывести на печать текущее окно.
	prompt()	Отображает диалоговое окно приглашения на ввод команды.
	releaseEvents()	Освобождает захваченные события указанного типа.
	resizeBy()	Изменяет размеры окна в соответствии с указанным смещением.
	resizeTo()	Изменяет размеры окна до указанных.
	routeEvent()	Передаёт события указанного типа на обработку собственными средствами.
	scroll()	Выполняет прокрутку документа в окне с указанной позиции.
	scrollBy()	Выполняет прокрутку документа в окне в соответствии с заданным смещением.

JS. Объекты браузера

	scrollTo()	Выполняет прокрутку документа к указанной позиции в окне.
	setHotKeys	Назначает или отменяет назначение комбинаций клавиш окна в случае отсутствия меню.
	setInterval()	Вызывает функцию или вычисляет выражение каждый раз по прошествии определенного числа миллисекунд
	setResizable()	Определяет, может ли пользователь изменять размеры окна.
	setTimeout()	Вызывает функцию или вычисляет выражение по прошествии определенного числа миллисекунд.
	setZOptions()	Позволяет определять z-порядок расположения элементов в окне.
	stop()	Останавливает загрузку элементов в текущем окне.
Свойство	closed	Определяет был ли экземпляр window закрыт.
	crypto	Обеспечивает доступ к возможностям кодирования Navigator.
	defaultStatus	Определяет заданное по умолчанию сообщение в строке состояния окна.
	document	Ссылается на всю информацию относительно документа, находящегося в данном окне. Обратитесь к описанию объекта Document для получения дополнительной информации.
	frames	Ссылается на всю информацию относительно фреймов данного окна. Обратитесь к описанию объекта Frame для получения дополнительной информации.
	history	Ссылается на URL-адреса, которые посетил данный пользователь.
	innerHeight	Содержит высоту в точках области отображения, занятой текущим окном.
	innerWidth	Содержит ширину в точках области отображения, занятой текущим окном.
	length	Определяет количество фреймов в текущем окне
	location	Содержит текущий URL страницы, загруженной в окно
	locationbar	Ссылается на окно адреса браузера.
	locationbar.visible	Содержит логическое значение, которое сообщает, видно ли пользователю окно адреса браузера.
	menubar	Ссылается на меню браузера.
	menubar.visible	Содержит логическое значение, которое сообщает, видно ли пользователю меню браузера.
	name	Содержит имя окна
	offscreenBuffering	Содержит логическое значение, которое позволяет определять, выполнялось ли обновление окна в экранном буфере.
	opener	Содержит имя окна, из которого было открыто вторичное окно.
	outerHeight	Содержит высоту в точках внешней области текущего окна.

JS. Объекты браузера

	outerWidth	Содержит ширину в точках внешней области текущего окна.
	pageXOffset	Содержит координату x текущего окна.
	pageYOffset	Содержит координату y текущего окна.
	parent	Ссылается на родительское окно, которое отображает текущий фрейм.
	personalbar	Ссылается на полосу <i>personal</i> браузера.
	personalbar.visible	Содержит логическое значение, которое сообщает, видна ли пользователю полоса прокрутки.
	screenX	Ссылается на координату левого верхнего угла окна браузера.
	screenY	Ссылается на координату левого верхнего угла окна браузера.
	scrollbars	Ссылается на строку состояния браузера.
	scrollbars.visible	Содержит логическое значение, которое сообщает видна ли пользователю строка состояния браузера.
	self	Ссылается на текущее окно.
	status	Ссылается на сообщение, отображаемое в строке состояния окна.
	statusbar	Ссылается на строку состояния браузера
	statusbar.visible	Содержит логическое значение, которое сообщает, видна ли пользователю строка состояния браузера.
	toolbar	Ссылается на панель инструментов браузера.
	toolbar.visible	Содержит логическое значение, которое сообщает, видна ли пользователю панель инструментов браузера.
	top	Ссылается на родительское окно, отображающее текущий фрейм.
	window	Ссылается на текущее окно.

Пример. Создание окна при помощи сценария.

```

<!DOCTYPE html>
<html>
<head>
<script>
function openWin() {
myWin= open("page.html", "newWindow",
"width=400,height=300,status=no,toolbar=no,menubar=no");
}
</script>
</head>
<body>
<a href="javascript:openWin()">Открыть новое
окно</a><br>
</body>
</html>

```

2 Объект document

Объект document соответствует всему гипертекстовому документу, вернее, той его части, которая заключена в контейнер `<body>...</body>`. Документы отображаются в окне браузера, поэтому каждый из них связан с определенным окном. Все HTML-объекты являются свойствами объекта document, поэтому они находятся в самом документе. Например, в языке сценариев JavaScript к первой форме документа можно обратиться, используя следующее выражение:

```
documents.forms[0]
```

В то время, как к первой форме во втором фрейме следует обращаться выражением, записанным ниже:

```
parent.frames[1].document.forms[0]
```

Для обращения к свойствам и методам объекта document применяется следующий синтаксис:

```
document.propertyName  
document.methodName(parameters)
```

2.1 Основные свойства объекта Document

Объект document имеет достаточно много свойств, каждое из которых соответствует определенному HTML-тегу в текущем документе, например:

- **alinkColor** соответствует атрибуту **alink** тега `<body>`;
- **anchors** – массив, который соответствует всем меткам в документе;
- **bgColor** соответствует атрибуту **bgColor** (цвет фона) тега `<body>`;
- **fgColor**
- **cookie** представляет собой фрагмент информации, записанный на локальный диск;
- **forms** – массив, содержащий все теги `<form>` в текущем документе;
- **images** – массив изображений, ссылки на которые заданы в текущем документе;
- **lastModified** – дата последнего изменения текущего документа;
- **linkColor** соответствует атрибуту **linkColor** (цвет гиперсвязи по умолчанию);
- **links** – массив, содержащий все гиперсвязи в текущем документе;
- **location** соответствует адресу URL текущего документа;
- **referrer** соответствует адресу URL документа, из которого пользователь перешел к текущему документу;
- **title** соответствует содержимому контейнера `<title>...</title>`
- **vlinkColor** соответствует атрибуту **vlinkColor** (цвет посещаемой связи) тега `<body>`

2.2 Основные методы объекта document

Метод **clear()** предназначен для очистки текущего документа. Лучше использовать для очистки методы **open()** и **close()**. Для записи информации в браузер применяют методы **write()** и **writeln()** (если находится в контейнере `pre`). Поскольку эти методы записывают текст в браузер в HTML-формате, можно создавать любой HTML-документ динамически, включая готовые приложения на языке JavaScript. Если в окно загружен документ, то

JS. Объекты браузера

запись данных поверх него может привести к сбою. Поэтому в окноследует записывать поток данных, для чего с помощью метода `document.open()` нужно открыть документ, а затем вызвав необходимое количество раз метод `document.write()`, записать данные в документ. В заключение, чтобы послать данные в браузер, следует вызвать метод `document.close()`

Методы и свойства объекта `document` представлены в таблице 3.

Таблица 3 – Методы и свойства объекта `document`

Тип	Элемент	Описание
Методы	<code>captureEvents()</code>	Захватывает события, которые будут обрабатываться документом.
	<code>close()</code>	Закрывает поток вывода документа.
	<code>contextual()</code>	Позволяет выборочно применить стиль к HTML-элементу, который появляется в определенном контексте.
	<code>getSelection()</code>	Возвращает выделенный текст.
	<code>handleEvent()</code>	Вызывает обработчик для указанного события.
	<code>open()</code>	Открывает поток вывода документа
	<code>releaseEvents()</code>	Освобождает события, захваченные документом.
	<code>routeEvent()</code>	Перенаправляет перехваченные события к другим объектам
	<code>write()</code>	Добавляет текст к документу.
	<code>writeln()</code>	Добавляет к документу текст и символ новой строки.
Свойства	<code>alinkColor</code>	Цвет активной ссылки.
	<code>all</code>	Массив всех HTML-дескрипторов документа.
	<code>anchors</code>	Массив объектов <code>Anchor</code> .
	<code>applets</code>	Массив объектов <code>Applet</code> .
	<code>bgcolor</code>	Цвет фона документа.
	<code>classes</code>	Массив классов таблиц стилей.
	<code>cookie</code>	cookie-набор, связанный с документом
	<code>domain</code>	Домен документа.
	<code>embeds</code>	Массив встроенных объектов.
	<code>fgcolor</code>	Цвет текста в документе.
	<code>forms</code>	Массив объектов <code>Form</code> .
	<code>formName</code>	Определяет экземпляр <code>Form</code> , к которому обращаются с использованием атрибута <code>name</code> в дескрипторе <code><form></code> .
	<code>height</code>	Определяет высоту документа в точках.
	<code>ids</code>	Массив идентификаторов таблиц стилей
	<code>images</code>	Массив объектов <code>Image</code> .
	<code>lastModified</code>	Дата последней модификации документа.
	<code>layers</code>	Массив объектов <code>Layer</code> .
	<code>linkColor</code>	Цвет ссылок.
	<code>links</code>	Массив объектов <code>Link</code> .
	<code>plugins</code>	Массив подключаемых модулей.
	<code>referrer</code>	URL-документа, с которым был связан текущий документ.
	<code>tags</code>	Массив дескрипторов таблиц стилей.
	<code>title</code>	Заголовок документа.
	<code>URL</code>	URL текущего документа.
	<code>vlinkColor</code>	Цвет посещенных ссылок
	<code>width</code>	Определяет ширину документа в точках.

2.3 Примеры использования объекта document

Для записи текста в HTML-формате в браузер иногда применяют функцию `document.writeln()`.

Например, можно динамически создавать теги изображений, выводя изображения на экран одним из следующих способов:

```
document.open();
document.writeln("<imgsrc=myimage.gif>");
document.close();
```

Спомощью JavaScriptпрограмм, в частности при помощи объекта `document`, можно создавать законченные HTML-документы и другие JavaScript программы. Например,

```
document.open();
document.writeln("<script
language='JavaScript'>"+"alert('Hello World!')"+"</script>");
document.close();
```

Следует заметить, что в приведенных примерах несколько строк объединяются при помощи операции сложения `+`. Этот способ удобно применять, когда строки текста программы слишком длинны, чтобы поместиться в редактируемом окне, или когда сложные строки необходимо разбить на несколько простых.

3 Объект Location

Данный объект сохраняет местоположение текущего документа в виде адреса URLэтого документа. При управлении объектом `location` существует возможность изменять адрес URL-документа. Объект `location` связан с текущим объектом `window` – окном, в которое загружен документ. Документы не содержат информации об адресах URL.

Синтаксис объекта `location`следующий:

```
[windowVar.]location.propertyName
```

где `windowVar`– необязательная переменная, задающая конкретное окно, к которому хотите обратиться. Эта переменная также позволяет обратиться к фрейму во фреймосодержащем документе при помощи свойства `parent` – синонима, используемого при обращении к объекту `window` верхнего уровня, если окон несколько. Объект `location`является свойством объекта `window`. Если обратиться к объекту `location` без указания имени окна, то подразумевается свойство текущего окна.

Свойство `location` объекта `window` легко перепутать со свойством `location` объекта `document`. Значение свойства `document.location` изменить нельзя, а значение свойства `location` окна – можно, например, при помощи выражения `window.location.property`. Значение `document.location` присваивается объекту `window.location` при первоначальной загрузке документа, потому что документы всегда загружаются в окна.

Свойства и методы объекта `location` представлены в таблице 4.

Таблица 4 – Методы и свойства объекта `location`

Тип	Элемент	Описание
Методы	<code>reload()</code>	перезагружает текущийURLв окне браузера.

JS. Объекты браузера

	<code>replace()</code>	Загружает новую страницу вместо текущей.
Свойство	<code>hash</code>	Представляет адре привязки в URL, которое начинается символом "#".
	<code>host</code>	Представляет имя а и номер порта, указанные в URL
	<code>hostname</code>	Имя хоста и домена (или цифровой IP-адрес) в адресе URL текущего документа.
	<code>href</code>	Представляет полный URL.
	<code>pathname</code>	Представляет часть PATH_INFO из URL (часть адреса URL, описывающая каталог, в котором находится документ)
	<code>port</code>	Представляет номер порта, указанный в URL.
	<code>protocol</code>	Представляет протокол, указанный в URL.
	<code>search</code>	Часть URL, касающаяся поиска, включая символ ?.
	<code>target</code>	Соответствует атрибуту target в теге <href>

Примеры использования объекта location

Чтобы присвоить свойству location текущего окна в качестве значения новый адрес URL, следует использовать такой вид:

```
self.location="http://wdstudio.al.ru";
```

который в данном случае загружает в текущее окно Web-страницу. Объект self можно опустить, поскольку он является ссылкой на текущее окно.

Чтобы загрузить ресурс в фрейм содержащий документ, можно записать так:

```
parent.frames[0].location="http://my.site.ru"
```

где `parent.frames[0]` соответствует первому фрейму в текущем документе.

4 Объект History

Объект history содержит список адресов URL, посещенных в этом сеансе. Объект history связан с текущим документом. Несколько методов этого объекта позволяют загружать в браузер различные ресурсы и обеспечивают навигацию по посещенным ресурсам.

Синтаксис объекта history следующий:

```
history.propertyName  
history.methodName (parameters)
```

Методы и свойства объекта history представлены в таблице 5.

Таблица 5 – Свойства и методы объекта history

Тип	Элемент	Описание
Метод	<code>back()</code>	Загружает предыдущий URL в списке хронологии
	<code>forward()</code>	Загружает следующий URL в списке хронологии.
	<code>go()</code>	Загружает URL из списка хронологии, расположенный по указанному смещению.
Свойство	<code>current</code>	Ссылается на текущий URL в списке хронологии.
	<code>length</code>	Возвращает количество элементов в списке хронологии
	<code>next</code>	Ссылается на следующий URL в списке хронологии
	<code>previous</code>	Ссылается на предыдущий URL в списке хронологии

JS. Объекты браузера

Примеры использования объекта history.

Чтобы посмотреть предыдущий загруженный документ следует воспользоваться следующим оператором:

```
history.go(-1);
```

или

```
history.back();
```

Для обращения к истории конкретного окна или фрейма применяют объект parent. Следующий оператор:

```
parent.frames[0].history.forward();
```

загружает в первый фрейм предыдущий документ.

Если открыто несколько окон браузера, можно использовать вид, представленный ниже.

```
window1.frames[0].history.forward();
```

В первый фрейм окна window1 будет загружен следующий документ из списка объектов history.

5 Объект Navigator

Объект navigator содержит информацию об используемой в настоящее время версии браузера. Этот объект применяется для получения информации о версиях браузера.

Синтаксис объекта navigator следующий:

```
navigator.propertyName
```

Методы и свойства объекта navigator представлены в таблице 6.

Таблица 6 – Свойства и методы объекта navigator

Тип	Элемент	Описание
Метод	javaEnabled()	Функция, проверяющая, поддерживается ли данный браузер Java.
	plugins.refresh()	Перестраивает массив установленных подключаемых модулей.
	preference()	Позволяет читать и устанавливать некоторые пользовательские предпочтения в браузере.
	taintEnabled()	Проверяет включена ли функция отсеивания нежелательных данных.
Свойства	appName	Представляет кодовое имя браузера.
	appVersion	Представляет официальное имя браузера.
	language	Выдает информацию о версии браузера.
	mimeType	Выдает язык браузера.
	mimeType	Ссылается на массив объектов MimeType, который содержит все поддерживаемые браузером типы MIME.

JS. Объекты браузера

	platform	Строка, указывающая на платформу, на которой выполняется браузер.
	plugins	Ссылается на массив объектов Plugin, который содержит все подключаемые модули браузера.
	userAgent	Строка, содержащая значение заголовка "user-agent". Кодовое имя и версия браузера.

События не определены для этого объекта. Да и свойства только для чтения, так как ресурс с информацией о версии недоступен для редактирования.

Внутренние объекты и объекты, связанные с тегами HTML рассмотрены в лабораторных работах.

6 Объект Global

Объект Global впервые появился, когда JavaScript 1.0 стал восприниматься в качестве стандарта. А потом компания Microsoft достаточно «внезапно» объединила высокоуровневые свойства и методы в объект Global, да и Netscape упомянула о нем в нескольких своих документациях. Вот так и появился объект Global – высокоуровневые свойства и методы и при этом отсутствие родительского объекта.

Объект Global имеет множество свойств и методов, которые перечислены в таблице1. Несложно убедиться, что большинство из них используется в сценариях JavaScript.

Таблица 1 – Методы и свойства объекта Global

Тип	Элемент	Описание
Методы	escape()	возвращает объект String, в котором все алфавитно-цифровые символы преобразовываются в их цифровые эквиваленты.
	eval()	Принимает строку с операторами JavaScript и тестирует ее на предмет корректности синтаксиса.
	isFinite()	Определяет, имеет ли переменная конечные границы.
	isNaN()	Определяет, содержит ли переменная допустимое число.
	parseFloat()	Выполняет преобразование строкового аргумента в число с плавающей точкой.
	parseInt()	Выполняет преобразование строкового аргумента в целое число.
	unescape()	Принимает шестнадцатичное значение и возвращает ASCII-эквивалент в кодовой таблице ISQ-Latin-1.
Свойства	Infinity	Ключевое слово, используемое для представления плюс бесконечности.
	NaN	Представляет объект «не число», не имеющий эквивалентов ни с одним из чисел.

Свойства и методы объекта Global обычно применяются без ссылки на объект.

Задания для самостоятельной работы

Составить программы с использованием одного-двух свойств и методов каждого из рассмотренных объектов